

Code Review Request

Software Engineering for Scientists

Overview

Title: Building the peptide mass database

Author: Ryan Layer

Reviewer(s): a classmate

Context

Hypedsearch identifies peptides from mass spectrometry data. A peptide is a short chain of amino acids cut from a protein. The instrument breaks each peptide into fragments and reports the mass-to-charge ratio (m/z) of each fragment.

Fragments that keep the start of the peptide are b ions. Fragments that keep the end are y ions. The data are noisy, with many false peaks and some missing ones.

To find the source protein, we precompute the m/z of every possible fragment in a database of known proteins. Observed peaks are then looked up in that table.

What this change does

These two functions build the lookup table from a protein FASTA file. Every k-mer of every protein gets one entry per ion type and charge.

Where to look

Review these in this order. Links are pinned to commit 3b8557e.

Function or file	What it does	Link
<code>max_mass</code>	m/z of a whole k-mer as a b or y ion	gen_spectra.py L49 to 60
<code>get_data</code>	database rows for one k-mer at charges 1 and 2	merge_search.py L47 to 58

How it works

For each function, give the inputs, the output and any assumptions.

`max_mass(sequence, ion, charge)`

- `sequence` an amino acid string such as "DPQVAQLELGG"
- `ion` 'b' or 'y'
- `charge` 1 (singly charged) or 2 (doubly charged)
- Returns the m/z of the full-length ion as a float. It sums the residue masses, adds the base mass for that ion type and charge, then divides by the charge.

`get_data(kmer, start, end, protein_num, ion)`

- `kmer` the peptide sequence

- start, end its position in the parent protein, 0-based
- protein_num the index of the parent protein in the FASTA file
- ion 'b' or 'y'
- Returns a list of two tuples, one per charge, each (mass, start, end, ion_int, charge, protein_num). ion_int is 0 for b and 1 for y.

How to run it

From src/, run

```
python -c "import gen_spectra as g; print(g.max_mass('DPQVAQLELGG', ion='b', charge=1))"
```

It should print 1108.563316435.

Tests none yet

Feedback I want

1. Is the charge handling correct? I want the doubly charged value to be the true m/z.
2. Is end inclusive or exclusive? I wrote "closed interval" in my notes, but I am not sure that matches the caller.
3. Would these functions be easy to test? What tests would you write first?

Out of scope

The database insert code and the clustering step. Both are changing next week.

Known issues

There is commented-out code near get_data that I have not cleaned up.

Thank you for taking the time to review this!